Module 6 Lab Session 3: Discoverability and Documentation

Module	M6 Building RESTful Web APIs
Session	3 of 3
Exercises	5 (HATEOAS Links via `LinkGenerator`), 6 (Scalar Documentation and Endpoint Metadata) + final lab checkpoint

Prerequisites Check Before You Start

Your TMS API must already have:

- The full Session 1 contract on `CoursesController` and `EnrollmentsController` (Exercises 1–3).
- The Session 2 paginated collection action with `PagedResponse<T>`, capped `PageSize`, and the filter-then-count-then-page sequence inside `CourseService.GetCoursesAsync` (Exercise 4).
- The global `AuditLogFilter` registered through `AddControllers(options => options.Filters.Add<AuditLogFilter>())`.
- The deterministic `DataSeeder` populating exactly 25 courses on Development startup.

If any item is missing, finish Session 2 before continuing. HATEOAS links sit on top of a working detail endpoint; Scalar metadata sits on top of endpoints that already return the right shapes.

What This Sessions Is Really About

Session 1 made the contract correct. Session 2 made it scale. Today the contract becomes **self-describing**.

Two questions the Angular team in M8 and any external integration partner is going to ask the moment they open `/scalar/v1`:

1. *"Given a course, what can I do with it?"* the answer is HATEOAS links in the detail response. Self, update, delete, list-enrolments, and conditionally a "create-enrolment" link the frontend uses to render or hide the Enrol button.

2. *"What does each endpoint accept and return?"* the answer is `[ProducesResponseType]`, `[EndpointSummary]`, `[EndpointDescription]`, and `[Tags]` decorating every action so Scalar can generate a real integration guide instead of a list of routes with no schemas.

Two short exercises then, and the rest of the lab time is for the **final checkpoint table** the eight-row verification at the end of this handout that proves every Session 1, 2, and 3 deliverable is green before the assessment.

Exercise 5: HATEOAS Links with `LinkGenerator` (LO 6.5)

Scenario: The Angular team in M8 will hard-code URLs like `/api/courses/${id}/enrollments` in TypeScript constants. The next time the backend team renames `enrollments` to `enrolments` (or moves it under `/api/registrations/{id}/courses`), every Angular component breaks. HATEOAS shifts that responsibility to the API: the response includes the URLs the client should call next, and the client follows links instead of constructing them.

A second realism point: even *inside* the API, you do not want to build links by string interpolation. `$"/api/courses/{id}"` is a counter-example, not a teaching path duplicate route information that breaks silently the day a route changes. Real .NET teams use `LinkGenerator` (or `IUrlHelper`) so links are derived from the same routing metadata the controller is already using.

Step 1 Define the link DTO

Create `Dtos/LinkDto.cs`:

```
namespace Tms.Api.Dtos;

public record LinkDto(string Href, string Rel, string Method);
```

Step 2 Define the detail DTO

Create `Dtos/CourseDetailDto.cs`. Note that `CourseDetailDto` is the **detail** shape it includes everything in `CourseResponseDto` plus the `Links` array. The list/page response keeps using `CourseResponseDto` (no per-item links a 50-item list with links per item is mostly noise):

```
namespace Tms.Api.Dtos;

public record CourseDetailDto
{
```

```
    public required int Id { get; init; }
    public required string Code { get; init; }
    public required string Title { get; init; }
    public required int MaxCapacity { get; init; }
    public required int EnrollmentCount { get; init; }
    public required IReadOnlyList<LinkDto> Links { get; init; }
}
```

Step 3 Wire `LinkGenerator` into the controller

`LinkGenerator` is a service the framework registers automatically you ask for it in the controller constructor. Update `CoursesController`:

```
using Microsoft.AspNetCore.Mvc;
using Microsoft.AspNetCore.Routing;
using Tms.Api.Dtos;
using Tms.Api.Services;

namespace Tms.Api.Controllers;

[ApiController]
[Route("api/courses")]
public class CoursesController(
    ICourseService courseService,
    LinkGenerator linkGenerator) : ControllerBase
{
    // ... existing actions ...
}
```

Step 4 Replace `GetCourseById` to return `CourseDetailDto` with links

This is the **graded** part the order of operations and the choice of `LinkGenerator.GetPathByName` (preferred when you have stable route names) over hand-rolled strings is exactly what the integrity lab inspects:

```
[HttpGet("{id:int}", Name = nameof(GetCourseById))]
public async Task<IActionResult> GetCourseById(int id, CancellationToke
n ct)
{
    var course = await courseService.GetByIdAsync(id, ct);
    if (course is null) return NotFound();

    // TODO 1: Use linkGenerator.GetPathByName(HttpContext, routeName,
values) to build each href.
    //         The route names are the ones you set with `Name = nameof
(...)` on the actions:
    //           - nameof(GetCourseById)           for /api/courses/{i
d}
    //           - nameof(GetEnrollment)           for /api/courses/{c
```

```
ourseId}/enrollments/{id}
    //         For the list-enrolments link, you will need to build the
 path manually with
    //         linkGenerator.GetPathByAction(HttpContext, action: "GetE
nrollments",
    //                                       controller: "Enrollments",
 values: new { courseId = id })
    //         OR add Name = "ListCourseEnrollments" on the [HttpGet] o
f EnrollmentsController and use GetPathByName.

    // TODO 2: Build a List<LinkDto> with these entries:
    //         - { rel: "self",        method: "GET",    href: GetCours
eById name with { id } }
    //         - { rel: "update",      method: "PUT",    href: GetCours
eById name with { id } }
    //         - { rel: "delete",      method: "DELETE", href: GetCours
eById name with { id } }
    //         - { rel: "enrollments", method: "GET",    href: <list-en
rollments path> }
    //         If course.EnrollmentCount < course.MaxCapacity, also add:
    //         - { rel: "enroll",      method: "POST",   href: <list-en
rollments path> }
    //         The conditional link is what makes HATEOAS earn its cost
  the Angular
    //         team uses its presence/absence to render or hide the Enr
ol button without
    //         duplicating the capacity rule in TypeScript.

    // TODO 3: Build a CourseDetailDto from `course` plus the Links lis
t and return Ok(detailDto).

    throw new NotImplementedException();
}
```

A note on null-handling: `GetPathByName` returns `string?` the path is null if the route name does not exist or the values do not match the route template. In production code you assert non-null after building (the route name is yours to control). For lab clarity, treat a null path as a bug to fix, not as a runtime branch the Lab Verification table at the end of the handout flags it if the JSON contains `"href": null`.

Step 5 Verify

Step	Action	Expected status	Expected body / observation
1	`GET /api/courses/1` (any seeded course not at	`200 OK`	Body has the five fields plus a `links` array of **5** entries (`self`,

Step	Action	Expected status	Expected body / observation
	capacity)		`update`, `delete`, `enrollments`, `enroll`)
2	Use the `enrollments` href from step 1 in a fresh GET request	`200 OK`	Returns the enrolment list for that course (you may need a `GetEnrollments` action on `EnrollmentsController` first see Step 6)
3	Fill the course to capacity (`POST /api/courses/{id}/enrollments` with valid `studentId` until `EnrollmentCount == MaxCapacity`)	`201 Created` (each call)	Each enrolment succeeds
4	`GET /api/courses/{id}` after step 3 fills capacity	`200 OK`	`links.length == 4` the `enroll` link is **gone**
5	Inspect any `href` in the response	(read)	Starts with `/api/courses/...` never `null`, never `${id}` literal

Step 6 Add the list-enrolments action you discovered you needed

Step 2 of verification reveals a gap: you have a route template for `GET /api/courses/{courseId}/enrollments/{id}` (the single enrolment) but not for `GET /api/courses/{courseId}/enrollments` (the list). Add it now to `EnrollmentsController`:

```
[HttpGet(Name = "ListCourseEnrollments")]
public async Task<IActionResult> GetEnrollments(int courseId, Cancellat
ionToken ct)
{
    // TODO 4: Confirm the parent course exists (courseService.GetByIdA
sync); 404 if not.
    //         Then return Ok(await enrollmentService.GetByCourseAsync
(courseId, ct))
    //         where GetByCourseAsync projects to a List<EnrollmentResp
onseDto>.
    throw new NotImplementedException();
}
```

You will need to add `GetByCourseAsync` to `IEnrollmentService` and `EnrollmentService`. The pattern matches Session 1's `GetByIdAsync` shape `AsNoTracking`, `Where`, `Select` to DTO, `ToListAsync(ct)`. With this in place, the

`enrollments` href from your `LinkGenerator` call resolves to a real endpoint, and Step 5's verification table passes end-to-end.

Troubleshooting

Symptom	Likely cause	Fix
`href` is `null` in the JSON	Route name does not exist or values do not match the template	Confirm the action has `Name = nameof(...)` and the `values` object names match the route parameters (e.g. `id`)
`href` is `/api/courses/{id}` (literal `{id}`)	You used string interpolation instead of `LinkGenerator`	Replace with `linkGenerator.GetPathByName(HttpContext, nameof(...), new { id })`
`enroll` link appears even when course is full	Conditional check missing	Wrap the `enroll` link push in `if (course.EnrollmentCount < course.MaxCapacity)`
`links` is `Links` in the JSON	Default JSON serialiser case is not camelCase	ASP.NET Core uses camelCase by default verify no custom `JsonSerializerOptions` overrides it
`enrollments` href returns 404 when followed	List-enrolments action does not exist yet	Add Step 6's `GetEnrollments` action and the supporting service method

Exercise 6: Scalar Documentation and Endpoint Metadata (LO 6.6)

Scenario: A potential integration partner opens `https://localhost:<port>/scalar/v1` and finds your endpoints listed with no descriptions, no response types, and no grouping. They email you asking what `POST /api/courses` returns on validation failure. The right answer is not "I will write you a doc" it is "open Scalar again, the metadata is there now". This exercise turns the live Scalar page into a real integration guide.

Step 1 Decorate `CoursesController` actions

Open `Controllers/CoursesController.cs`. Add the metadata attributes to the three actions you already have. The `[Tags("Courses")]` goes on the controller class so all course endpoints group together in Scalar:

```
[ApiController]
[Route("api/courses")]
[Tags("Courses")]
[Produces("application/json")]
[ProducesResponseType(typeof(ProblemDetails), StatusCodes.Status500Inte
rnalServerError)]
public class CoursesController(
    ICourseService courseService,
    LinkGenerator linkGenerator) : ControllerBase
{
    [HttpGet]
    [ProducesResponseType(typeof(PagedResponse<CourseResponseDto>), Sta
tusCodes.Status200OK)]
    [EndpointSummary("List courses with pagination")]
    [EndpointDescription("Returns a paginated, optionally filtered list
 of TMS courses. PageSize is capped at 50.")]
    public async Task<IActionResult> GetCourses(
        [FromQuery] PagedRequest request, CancellationToken ct) { /* un
changed */ }

    [HttpGet("{id:int}", Name = nameof(GetCourseById))]
    [ProducesResponseType(typeof(CourseDetailDto), StatusCodes.Status20
0OK)]
    [ProducesResponseType(typeof(ProblemDetails), StatusCodes.Status404
NotFound)]
    [EndpointSummary("Get a course by ID")]
    [EndpointDescription("Returns course details with HATEOAS links. Re
turns 404 if the course does not exist.")]
    public async Task<IActionResult> GetCourseById(int id, Cancellation
Token ct) { /* unchanged */ }

    [HttpPost]
    [ProducesResponseType(typeof(CourseResponseDto), StatusCodes.Status
201Created)]
    [ProducesResponseType(typeof(ValidationProblemDetails), StatusCodes.
Status400BadRequest)]
    [ProducesResponseType(typeof(ProblemDetails), StatusCodes.Status409
Conflict)]
    [EndpointSummary("Create a new course")]
    [EndpointDescription("Creates a course with a unique code. Returns
409 if the course code already exists.")]
    public async Task<IActionResult> CreateCourse(
        CreateCourseRequest request, CancellationToken ct) { /* unchang
```

```
ed */ }
}
```

A few production habits visible here. `[Produces("application/json")]` declares the response content type so Scalar shows the right `Accept` and `Content-Type` headers in its "Try It" panel. The class-level `[ProducesResponseType(typeof(ProblemDetails), StatusCodes.Status500InternalServerError)]` declares the catch-all for unhandled exceptions every action inherits it, every Scalar entry shows that 500 returns `ProblemDetails`. And `[Tags("Courses")]` at the class level keeps the actions grouped applying `[Tags]` per-action would un-group them by accident.

Step 2 Decorate `EnrollmentsController` actions

```
[ApiController]
[Route("api/courses/{courseId:int}/enrollments")]
[Tags("Enrollments")]
[Produces("application/json")]
[ProducesResponseType(typeof(ProblemDetails), StatusCodes.Status500Inte
rnalServerError)]
public class EnrollmentsController(
    ICourseService courseService,
    IEnrollmentService enrollmentService) : ControllerBase
{
    [HttpGet(Name = "ListCourseEnrollments")]
    [ProducesResponseType(typeof(IReadOnlyList<EnrollmentResponseDto>),
 StatusCodes.Status200OK)]
    [ProducesResponseType(typeof(ProblemDetails), StatusCodes.Status404
NotFound)]
    [EndpointSummary("List enrolments for a course")]
    public async Task<IActionResult> GetEnrollments(int courseId, Cance
llationToken ct) { /* unchanged */ }

    [HttpGet("{id:int}", Name = nameof(GetEnrollment))]
    [ProducesResponseType(typeof(EnrollmentResponseDto), StatusCodes.St
atus200OK)]
    [ProducesResponseType(typeof(ProblemDetails), StatusCodes.Status404
NotFound)]
    [EndpointSummary("Get one enrolment for a course")]
    public async Task<IActionResult> GetEnrollment(int courseId, int id,
 CancellationToken ct) { /* unchanged */ }

    [HttpPost]
    [ProducesResponseType(typeof(EnrollmentResponseDto), StatusCodes.St
atus201Created)]
    [ProducesResponseType(typeof(ValidationProblemDetails), StatusCodes.
Status400BadRequest)]
    [ProducesResponseType(typeof(ProblemDetails), StatusCodes.Status404
```

```
NotFound)]
    [ProducesResponseType(typeof(ProblemDetails), StatusCodes.Status409
Conflict)]
    [EndpointSummary("Enrol a student in a course")]
    [EndpointDescription("Returns 404 if the course does not exist, 409
 if the course has reached MaxCapacity.")]
    public async Task<IActionResult> EnrollStudent(
        int courseId, EnrollStudentRequest request, CancellationToken c
t) { /* unchanged */ }
}
```

Step 3 Verify

Open `https://localhost:<port>/scalar/v1`. The verification is visual five things should be true on the page:

1. The left-hand index shows two collapsible groups: **Courses** and **Enrollments**.
2. Expand `POST /api/courses`. The summary reads "Create a new course". The "Responses" panel lists `201 Created` with a `CourseResponseDto` schema, `400 Bad Request` with a `ValidationProblemDetails` schema, and `409 Conflict` with a `ProblemDetails` schema.
3. Expand `GET /api/courses/{id}`. The summary reads "Get a course by ID". The "Responses" panel lists `200 OK` with `CourseDetailDto` (schema includes the `links` array), and `404 Not Found` with `ProblemDetails`.
4. Click "Try It" on `GET /api/courses/{id}` with `id = 1`. Scalar should pre-fill the URL, send the request, and show the JSON response inline. The `links` array is visible and follows the rules from Exercise 5.
5. The `POST /api/courses/{courseId}/enrollments` page lists all four response codes (`201`, `400`, `404`, `409`) that single endpoint is the most documented one in the API for a reason.

If your screenshot of Scalar matches those five points, the documentation is the integration guide. If anything is missing a status code, a schema, a tag you know which attribute is missing and on which action.

Troubleshooting

Symptom	Likely cause	Fix
Scalar shows endpoints with no summary	`[EndpointSummary]` not added	Add it to each action; the `using Microsoft.AspNetCore.Http;` namespace must be imported
Endpoints not	`[Tags]` on actions instead of the	Move `[Tags("Courses")]` to the class

Symptom	Likely cause	Fix
grouped flat list	controller class	level; remove any per-action `[Tags]`
`400` schema shows `ProblemDetails` not `ValidationProblemDetails`	Wrong type in the attribute	Use `typeof(ValidationProblemDetails)` for the `400` status they have different shapes in the JSON
`Try It` panel returns CORS errors	Browser blocked the cross-origin call	If you serve Scalar on the same origin, this should not happen. M8 covers CORS for now, run the API and Scalar from the same `https://localhost:<port>`
`409` not listed under `POST /api/courses` in Scalar	`[ProducesResponseType(typeof(ProblemDetails), StatusCodes.Status409Conflict)]` not added	Add it to the action
`500` declared but never appears in the response panel	That is fine `500` is the catch-all	Scalar shows it once on the controller block; individual actions do not need to repeat it

Final Lab Checkpoint All Six Exercises

Run this table from top to bottom against your live API. Every row must pass. The integrity lab grading mirrors these exact checks; if they pass here, the assessment is recording the same evidence.

#	Check	Pass means	Lab
1	`POST /api/courses` with valid body	`201 Created`, `Location` header points at `GetCourseById`, response is `CourseResponseDto`	S1 / E1
2	`POST /api/courses` with `{}`	`400 Bad Request`, body is `ValidationProblemDetails` with errors for `Code`, `Title`, `MaxCapacity`	S1 / E2
3	`POST /api/courses` with a duplicate code	`409 Conflict`, body is `ProblemDetails` with `Title = "Course code already exists"`	S1 / E3

#	Check	Pass means	Lab
4	`POST /api/courses/{id}/enrollments` into a course at capacity	`409 Conflict`, body is `ProblemDetails` with `Title = "Course is full"`	S1 / E3
5	`GET /api/courses/9999`	`404 Not Found`	S1 / E1
6	`GET /api/courses?page=2&pageSize=10`	`PagedResponse<CourseResponseDto>` with `totalCount == 25`, `totalPages == 3`, items.length ≤ 10	S2 / E4
7	`GET /api/courses?pageSize=9999`	Response `pageSize == 50` (capped)	S2 / E4
8	EF SQL log on a paged GET	One `SELECT COUNT(*)` and one paged `SELECT … LIMIT … OFFSET …` exactly two statements	S2 / E4
9	Console log on any API call	Two lines from `AuditLogFilter`: `TMS API call: …` and `TMS API response: …`	S2 / E4
10	`GET /api/courses/{id}` for a course not at capacity	`CourseDetailDto` with `links.length == 5` (`self`, `update`, `delete`, `enrollments`, `enroll`)	S3 / E5
11	`GET /api/courses/{id}` for a course at capacity	`CourseDetailDto` with `links.length == 4` (no `enroll` link)	S3 / E5
12	Any link `href` in the response	Starts with `/api/courses/...`; never `null`; never `${id}`	S3 / E5
13	`/scalar/v1` in the browser	Two grouped sections (Courses, Enrollments); every endpoint has summary + response schemas + status codes	S3 / E6
14	`POST /api/courses` row in Scalar	Lists `201 Created` (`CourseResponseDto`), `400 Bad Request` (`ValidationProblemDetails`), `409 Conflict` (`ProblemDetails`)	S3 / E6

If all fourteen rows pass, you have built a production-quality REST API contract: predictable routes, validated DTOs, business-rule status codes, mandatory bounded collections, hyperlinked discovery, and a documentation page that

earns its place. The Facilitator Answer Key contains reference solutions for cross-checking. Move on to the M6 Assessment Pack.

Why this session ends here and what M7 brings

You finish M6 with the **interface** of the TMS API solid: every endpoint has a name, a contract, a documented set of response shapes, and a self-describing JSON body that tells the client what to do next. The next four problems **versioning** (when the contract has to change without breaking existing clients), **caching** (when a `GET` does not have to hit the database every time), **rate limiting** (when a hostile or buggy client hammers the same endpoint), and **CQRS** (when read paths and write paths have genuinely different shapes) all assume this contract is in place. M7 is about hardening it; the contract itself is done here.